OnboardAI Codebase Guide

This guide walks through every file in the project and explains what it does and why it was written that way.

It is split into two parts on purpose. Part 1 covers the code that produces the work being assessed at the mid term review, which is everything up to data splitting. Part 2 covers the rest of the system, which was built ahead of schedule and belongs to the final review.

If you are preparing for the mid term review, Part 1 is the part you need to be able to explain. Part 2 is useful background and is worth reading, but no mark depends on it at this stage.

Part 1: Code Covered by the Mid Term Rubric

The rubric awards marks for problem definition, data understanding, preprocessing, exploratory analysis, data splitting, scope, and teamwork. The files below produce all of that.

1.1 Data generation

`backend/ml/synth.py`

Generates the onboarding document corpus. This is the dataset the document classifier will be trained on in the next phase.

It holds 21 templates spread across 9 document classes, which are offer letters, government identity documents, safety certifications, bank mandates, background check consents, intellectual property agreements, education certificates, professional licences, and a final class called `other` for files that satisfy no requirement at all.

The important function is `generate()`. For each sample it picks a template, fills the placeholder slots from pools of names, dates, identifiers and organisations, and then degrades the result.

Degradation is the part worth understanding. The function `_degrade()` applies one of three tiers:

- `clean` leaves the document almost intact, with light character noise applied to about a

third of samples.

- `degraded` strips the header about half the time, bleeds in a sentence belonging to a

different document class, and applies three to nine per cent character noise.

- `severe` always strips the header, cuts the document down to a short fragment, adds one or

two cross class sentences, and applies ten to twenty per cent character noise.

The mix is set by `DIFFICULTY_MIX` and is roughly forty per cent clean, thirty eight per cent degraded and twenty two per cent severe.

The reason for all of this is a lesson we learned the hard way. Our first version of this file had no degradation tiers, and the classifier trained on it scored a perfect 1.000 on every metric with zero cross validation variance. That is not a good model, it is a broken benchmark. The classes were separable on vocabulary alone. We rebuilt the generator until the severe tier produced a score we could believe.

`_add_noise()` simulates OCR damage using a list of character confusions that real OCR engines actually make, such as reading a zero as a capital O or the letter pair `rn` as an `m`.

`backend/ml/text_synth.py`

Generates the employee message corpus. One corpus, two labels: what the person wants (`intent`, 8 classes) and how they feel (`sentiment`, 3 classes).

Two mechanisms exist specifically to stop the problem being too easy.

`_VAGUE` holds messages such as "any update?" and "still waiting". These are assigned the intent the person actually had, but the text itself does not reveal it. This produces error that cannot be removed by a better model, which is correct, because the same thing happens with real messages.

`_BLEND_CLAUSES` appends a clause belonging to a second intent to about a quarter of messages, so a message about documents might also ask about leave. The label stays with the dominant intent.

The sentiment side has a subtlety that caused a real bug. Both the positive and the neutral prefix pools originally contained an empty string, which meant a message could be labelled positive while containing no

positive signal at all. That is mislabelled training data. The current code guarantees a sentiment marker for positive and negative samples unless the intent is `escalation` or `smalltalk`, where the body carries the tone on its own. A further fourteen per cent of negative messages get a polite opener such as "Thanks for getting back to me, but", which creates genuinely mixed signals.

`backend/ml/risk_synth.py`

Generates the tabular dataset of onboarding journeys, 6000 rows and 12 numeric features, with a binary target called `stalled`.

This file is the one most worth explaining in a viva, because it is not a simple random sampler. It is a latent variable process.

Three hidden traits are drawn for each simulated employee: `engagement`, `friction` and `support`. Every observable feature is then derived as a noisy measurement of those traits. For example `days_since_last_activity` is drawn from an exponential distribution whose rate depends on engagement and friction, and `avg_sentiment` is a noisy tanh of a weighted combination of all three.

The outcome is computed from the same hidden traits plus a term called `unobserved`, which no feature measures. That term is deliberate. It means no feature set can predict the outcome perfectly, which is what we want, because a model scoring near 1.0 on this data would prove the generator had leaked the answer rather than that the model was good.

Two features, `response_latency_hours` and `is_remote`, were removed from the feature list part way through the project. They are still simulated and still influence the outcome, but they are no longer offered as inputs. The reason is in Part 2 under `risk_model.py`.

1.2 Analysis and reporting

`backend/ml/data_report.py`

Produces the dataset card, the quality checks, the preprocessing evidence and the split audit. Run it with `python -m ml.data_report` from the `backend` directory. It writes `ml/artifacts/data_report.md` and a matching `.json`.

Key functions:

- `text_corpus_stats()` computes sample counts, class counts, balance ratio, length statistics,

exact duplicates, near duplicates after normalising case and whitespace, and empty records.

- `tfidf_evidence()` produces the before and after numbers for vectorisation, including the raw

token count, the feature count at `min_df=1` and at `min_df=2`, the matrix shape and the sparsity.

- `split_audit()` performs the stratified split and then measures how far each class proportion

drifted, and how many records appear in both folds.

- `numeric_audit()` produces per feature statistics, missing value counts, interquartile range

outlier counts, scaling evidence, the correlation matrix and correlation with the target.

- `leakage_impact()` is the most interesting one and is explained below.

`leakage_impact()` exists because we found duplicate messages appearing in both folds and assumed this was inflating our scores. Instead of assuming, the function trains the sentiment classifier once, splits the test set into the rows that also appear in training and the rows that do not, and scores both separately. The result contradicted us. Accuracy on the leaked rows is lower than on the clean rows, because the duplicated messages are the short ambiguous ones which are hard to classify anyway. The leakage is real but its effect is negligible.

The whole report is generated, never typed. If the data changes, the report changes with it.

`backend/ml/eda.py`

Produces ten figures and the observations that go with them. Run with `python -m ml.eda`.

The figures are a class distribution bar chart, a length histogram with a box plot by tier, a tier composition chart, intent and sentiment distributions, an intent by sentiment heatmap, the feature correlation matrix, stall rate by department, sentiment against outcome as both a box plot and overlaid histograms, a grid of all twelve feature distributions, and a scatter of progress against inactivity coloured by outcome.

The observations are computed in code rather than written by hand. For example, the correlation figure's note finds the strongest correlated pair at runtime and states its value. This means the commentary cannot drift away from the figure, which is a problem we had when the first version of the notes was hardcoded and the data was then regenerated.

The most useful finding from these figures is that `stage`, `progress` and `pct_tasks_complete` correlate above 0.9 with each other. Three features are measuring almost the same thing, which explains why removing any one of them changes an overall score very little.

1.3 Model training scripts

These produce models, which the mid term rubric excludes from marking. They are listed here because the split and preprocessing code lives inside them.

`backend/ml/train_classifier.py`

`build_features()` is the part relevant to the rubric. It returns a `FeatureUnion` of two vectorisers: word level TF-IDF over one and two grams, and character level TF-IDF over three to five grams within word boundaries. The character features exist so that the representation survives OCR damage that destroys whole words, which is exactly what the severe tier produces.

The split is 75 and 25, stratified, with `random_state=42`.

`backend/ml/train_nlp.py`

Trains intent and sentiment from the same corpus with the same pipeline shape. Both use logistic regression with balanced class weights, because sentiment is imbalanced by design.

`backend/ml/train_risk.py`

Compares logistic regression against histogram gradient boosting and keeps whichever scores higher on held out ROC AUC. It also runs the ablation study, which retrains with one feature removed at a time and reports the change. This is where the evidence that the NLP layer feeds the ML layer comes from: removing `avg_sentiment` costs about 0.0075 ROC AUC.

1.4 The test suite

`backend/tests/`

Thirty four test cases expanding to fifty five checks. Run with `python -m pytest tests -q`.

- `conftest.py` holds the fixtures. The important one is `tmp_storage`, which redirects file

storage to a temporary directory so that no test writes to the real document store.

- `test_data_quality.py` holds TC-01 to TC-08, covering completeness, balance, types, duplicate

counts and feature domains.

- `test_preprocessing_and_split.py` holds TC-09 to TC-15, covering vectoriser output, pruning,

scaling, stratification, fold overlap and reproducibility.

- `test_pipeline_and_safety.py` holds TC-16 to TC-26, covering the document pipeline contract and all the boundary and exceptional input cases.
- `test_nlp_retrieval_agents.py` holds TC-27 to TC-34, covering intent routing, sentiment, PII redaction, retrieval access control, agent tool boundaries and performance.
- `make_test_report.py` runs the suite and writes a report where the actual results come from a real run rather than being typed in.

Two tests, TC-05 and TC-14, assert that a known defect is present and bounded. They fail if the defect grows worse, and they fail if somebody fixes it without updating the documentation. That is intentional.

Part 2: The Rest of the System

Everything below was built ahead of the mid term schedule. None of it is assessed now.

2.1 Application skeleton

`backend/main.py`

The FastAPI application. It loads the environment file using a path anchored to its own location rather than the working directory, because the server is normally launched from the project root and a bare `load_dotenv()` looks in the wrong place and silently finds nothing. This caused a real bug where the API key appeared to be missing.

It also exposes `/api/health`, which actively contacts the configured model provider rather than only checking that an environment variable is set.

`backend/database.py`, `models.py`, `schemas.py`, `seed.py`

Standard SQLAlchemy setup. The database path is anchored to the file rather than the working directory, for the same reason as above.

`models.py` holds six original tables plus two we added. `Event` is a structured machine readable record of everything that happens, kept separate from `AuditLog`, which holds prose for the human readable timeline. The risk model reads features only from `Event`, so any action that does not write an event is invisible to the machine learning layer.

`DocumentUpload` keeps two columns apart on purpose. `verdict` is what the model concluded and `hr_decision` is what a human concluded. The disagreement between them is the training signal for improving the classifier later, which is lost if one overwrites the other.

`seed.py` computes start dates relative to today rather than hardcoding them. The original fixture had dates that had drifted into the future, which made the interface say somebody started on a date that had not happened yet and collapsed the elapsed time feature to its floor for every employee.

`backend/llm_provider.py`

The only module in the project that names a model vendor. `get_llm()` returns a chat model chosen by the `LLM_PROVIDER` environment variable, with the import deferred so an uninstalled provider never breaks the default path.

This was not a speculative abstraction. When the Gemini free tier returned a quota error during testing, switching to a local model took one environment variable and no code change.

`backend/storage.py`

File storage for uploads. Three details matter.

The declared content type from the browser is ignored. The real file type is identified from the leading magic bytes, so an HTML file labelled as a PDF is rejected.

Stored filenames are generated UUIDs, never the name the user supplied, which removes an entire class of path manipulation problems.

`resolve()` refuses to return a path that escapes the storage root, which is tested by TC-24.

`backend/events.py`

A single helper for writing to the event stream. Everything funnels through it so that nothing can quietly skip being recorded.

2.2 Document intelligence

`backend/ml/extract.py`

Turns a file into text. It prefers the embedded text layer of a PDF, which is both faster and more accurate than OCR, and falls back to OCR only when there is no text layer. OCR is an optional dependency, so the pipeline runs on a machine with no OCR engine installed and reports honestly that it could not read the image rather than crashing.

`backend/ml/classifier.py`

Loads the trained document classifier and returns a label with a confidence and the full score distribution. Loading is lazy and guarded, so a missing artefact degrades to a null prediction rather than an exception.

`backend/ml/fields.py`

Rule based extraction of dates, names and identifiers.

The date logic is the part worth reading. Identity documents routinely print the issue date and the expiry date on the same line. An earlier version searched the whole line for keywords, found the word "expiry" somewhere on it, and assigned the first date to it. That marked a valid passport as expired. The current `_nearest_role()` assigns each date the role of the label closest to its left, and falls back to a label on the preceding line.

The certificate number pattern had a similar fault. The abbreviation `cert` matched the opening of the word CERTIFICATE in a document title and captured `IFICATE` as the identifier. The pattern now requires the words "no", "number" or "ref" to follow.

`backend/ml/validate.py`

Applies the rules: does the document type match the requirement, is it in date, does the name match, is it signed where a signature is required. Each check carries a severity of blocking, warning or info.

`backend/ml/pipeline.py`

Chains the above into extract, classify, extract fields, validate, decide.

The decision policy is deliberately cautious. A blocking failure rejects. Confidence below 0.60 goes to a human. Any warning goes to a human. Only a confident classification with entirely clean checks approves automatically. Approving a bad document wrongly costs far more than asking a person to spend thirty seconds looking at it.

2.3 Natural language processing

`backend/ml/nlp_models.py`

Loads the intent and sentiment classifiers and returns both for a message. Sentiment is emitted as a continuous score of positive probability minus negative probability, which is smoother than the hard label and is what the risk model consumes.

`INTENT_CONFIDENCE_FLOOR` is 0.45. Below it nothing is routed to a specialist, because a vague message genuinely does not identify one.

`backend/ml/pii.py`

Strips email addresses, phone numbers, Aadhaar numbers, PAN, IFSC codes, passport numbers, account numbers and card numbers before any text is sent to a model provider. Patterns are ordered most specific first so that a generic long number rule cannot partially consume an IFSC code. Values are replaced with typed placeholders and are never returned, so they cannot leak into logs.

`backend/ml/rag.py`

Retrieval over the policy corpus in `backend/knowledge/`.

Scoring is hybrid, half TF-IDF cosine and half the cosine of an LSA projection of the same matrix. The lexical half catches exact policy wording and the semantic half catches paraphrase. A measured example: the question "can I take time off before my probation ends" scores 0.339 lexically against the relevant section but 0.846 semantically.

LSA was chosen over a neural embedding model so that the system runs offline with no model download, which mattered because the development machine could not reliably download large files during the project.

Retrieval is access controlled. A chunk is visible only if its organisation matches and, where the document restricts one, the employee's department is permitted.

Two thresholds exist. Below `RELEVANCE_FLOOR` nothing is returned. Between that and `WEAK_MATCH_SCORE` the passages are returned but flagged to the model as possibly irrelevant, and the question is recorded as a documentation gap. The second threshold was added after testing showed the common failure is a topically adjacent match rather than an empty result.

2.4 Prediction and explanation

`backend/ml/risk_model.py`

Builds the feature vector for a real employee from the database and the event stream, scores it, and explains the score.

Explanations are exact rather than sampled. For a standardised linear model, the coefficient multiplied by the scaled feature value is the SHAP value, so the attribution is deterministic and needs no sampling.

This file contained the most serious defect we found. Two features, `response_latency_hours` and `is_remote`, were hardcoded to fixed values at prediction time while the model had been trained on values that varied. That is train and serve skew: two learned coefficients were being applied to information the model never actually received. Neither value could be recovered from our data model, so both features were removed from the feature set and the model was retrained. The reported score barely changed, which is the point. It now describes what the model really sees.

2.5 Agents

`backend/agents/specialists.py`

Six specialist agents, each with its own system prompt and its own tool subset. The boundaries are enforced by construction. The Policy agent is never handed the tool that completes a task, so no prompt can talk it into doing so.

All tools are built inside a closure over the authenticated employee identifier, which is never a model supplied argument. This is an authorisation boundary rather than an instruction, and it held when we tested it with a direct prompt injection attempt.

`backend/agents/chatbot.py`

Routes a message to a specialist and runs it.

Routing uses the trained intent classifier rather than asking a language model which agent to use. This is one model call cheaper, deterministic and therefore testable, and its accuracy can be measured against a confusion matrix.

Specialists may transfer once if a message reaches the wrong one, which covers the classifier's residual error without risking a loop. Conversation threads are keyed per specialist, because replaying one agent's

tool calls into an agent that does not hold those tools is unsafe.

`_as_text()` exists because newer models return message content as a list of typed blocks, some of which are internal reasoning. Naively converting that to a string would have shown the model's reasoning to employees.

2.6 API routes

- `routes/api.py` holds employee and organisation endpoints, the stage override, the risk

endpoints and the knowledge gap report.

- `routes/documents.py` holds upload, retrieval, the analysis view, the HR decision endpoint

and the model agreement statistics.

- `routes/chat.py` runs the NLP layer, redacts personal data, calls the agent layer and records

the event.

2.7 Frontend

`src/` holds a React application with two role based views. `context/AppContext.jsx` holds all API calls and shared state. `components/DocumentAnalysis.jsx` renders the model's verdict, confidence and extracted fields. `components/RiskPanel.jsx` renders the risk score with its contributing factors.

One fix worth noting: `currentUser` was originally a snapshot taken at login and never refreshed, so completing a task updated the database and the employee's own screen showed nothing. The context now re syncs it whenever the employee list is refetched.

Commands Reference

Run everything from the `backend` directory with the virtual environment active.

```
python -m ml.data_report         # dataset card, quality checks, split audit
python -m ml.eda                 # ten figures and their observations
python -m pytest tests -q        # 55 checks
python -m tests.make_test_report # test report with real results
python -m ml.train_classifier    # retrain the document classifier
python -m ml.train_nlp           # retrain intent and sentiment
python -m ml.train_risk          # retrain the risk model with ablations
python -m ml.knowledge_seed      # write the policy corpus
python -m ml.make_demo_docs      # generate the seven demonstration PDFs
python seed.py                   # reset the database
```

From the project root:

```
python docs/build_midterm_deck.py   # the presentation
python docs/build_paper.py          # the IEEE paper
python docs/build_role_docs.py      # Tester and QA documents
```